

3+ years building in Unity — one shipped commercial title on Steam, a game-jam release, and a long list of solo projects, including a multiplayer tank game with hand-rolled netcode (prediction, reconciliation, lag compensation). On **Armageddonica** I split my time between gameplay/bugfixing and editor tooling for artists and designers, plus doing a lot of the unglamorous cross-team troubleshooting nobody assigns but somebody has to do. I pitch ideas when I have them and I'm usually right about whether a mechanic will hold up after a month of development.

PROFESSIONAL EXPERIENCE

Gameplay Programmer — Thrones of Ashes

Oct 2025 – Jul 2026 · Remote

Unreleased RPG — lore-driven world, full gear system (armor pieces with individual stats and enchantments, weapons with distinct effects), varied enemies and game modes. Team started at 8: 3 programmers, 3 artists, 2 game designers, a community manager.

- Refactored a sizable legacy codebase while shipping new features across it — the gear and stats system, weapon effects, enemy variety, and game modes.
- Designed and built the game's cutscene system, then built an editor tool on top of it so anyone on the team could put a cutscene together without touching code.
- Worked closely with the animator to get combat and cutscene animations wired up correctly, and implemented a steady stream of assets coming in from the art team.
- The programming team shrank from 3 to 2 to just me over the project's lifetime — I ended up carrying the engineering side solo for the last stretch.

Gameplay & Tools Programmer — neoTemplar

Feb 2025 – Sep 2025 · Remote

*Shipped **Armageddonica** — hybrid hack-and-slash ARPG + RTS, direct hero combat combined with large-scale autonomous army command. Team of 11: 3 programmers, 2 artists, a UI artist, a level designer, 2 game designers, a community manager, and the founder.*

store.steampowered.com/app/3166960/Armageddonica

- Owned day-to-day gameplay feature work and bug fixing on a live, shipping codebase — small fixes, bigger features, whatever came up; implemented UI from Figma handoffs.
- Turned our prefab-scattering tool into something the art team actually reached for daily: smarter placement logic so props stopped clumping together, a UI cut down to a few preset buttons, and a lighter version just for level designers. Also built a stack of small utilities around it — a screenshot tool, a null-prefab cleanup, a batch prefab replacer — nothing flashy, just things that saved people a few minutes every day.
- Replaced multi-step manual setups with one-click editor tools — cloud shadows, minimap generation, terrain grass presets — so designers stopped needing a walkthrough every time they wanted one.
- Ended up as the unofficial fixer for artists' technical problems — broken Git merges, missing shadows, terrain you couldn't click on. Happened often enough that people just started coming to me by default instead of opening a ticket.
- Noticed the game had no loading screen — scene changes just looked frozen — so I built one (random art/tips, fade transitions), then followed it up with a UI polish pass: hover states, press feedback, sound cues.
- Wrote a tool that batch-converts old Text components to TextMeshPro without losing formatting, and moved a lot of tunable gameplay values into ScriptableObjects so designers could tweak numbers themselves.
- Two weeks into the project, before I knew the whole game, a designer asked me to play through a chapter and report where it got confusing. I recorded myself doing it instead of writing notes — turned out to land a lot better with the team than a report would have.

SELECTED PERSONAL PROJECTS

Multiplayer Tank Game [Work in progress](#)

Competitive multiplayer shooter with custom netcode built on Mirror: client-side prediction, movement reconciliation, and lag compensation. Modular tank customization (hull / turret / commander) with unique turret ability kits (charge-based buffs, flamethrower, grenade launcher, beam-damage turret, AoE splash). Config-driven game modes (DM / TDM / CTF) implemented as ScriptableObjects wired through Zenject bindings.

Hermit Night 1 & 2 [Narrative horror](#)

itch.io/hermit-night · hermit-night-2

Two-part atmospheric horror built solo end-to-end: custom cutscene sequencing system, dynamic lighting/particle work for storm effects, and radio/audio-driven narrative pacing.

Locked In Nonsense [Comedic horror / exploration](#)

itch.io/locked-in-nonsense

Patrol/chase AI, multi-quest structure, and a full custom item system — including an in-world playable video player object and camera-based room monitoring.

Interpressure [Game jam — city builder](#)

itch.io/interpressure

Built with a 4-person team (2 programmers) for a "Bubbles"-themed jam: resource-driven building system, session save/load, 2D cutscene, and an in-game illustrated tutorial.

shootemup-64 [Rhythm-based arcade shooter](#)

github.com/ggStrider/shootemup-64

Music-synced color-matching shooter extended with a purchasable power-up economy (piercing shots, slow/haste zones) and post-run stats.

AnExprmnt [Comedic horror](#)

Built solo, purely to make friends laugh. Patrol/chase AI with periodic aggro, a 3-item ability kit driving a small risk/reward combat loop, and a "smash-button" interaction mechanic.

dhCDNAME [Narrative horror](#)

[itch.io/dhcdname](#)

First shipped project: mini-quest/objective tracking, interactable notes, and event-driven lighting/audio triggers for a thunderstorm sequence.

slowmo kenr [Bullet-hell arena](#)

Built around a time-slow mechanic with camera-zoom feedback — waves of attacks (homing rockets, sinusoidal bullet patterns, lasers, timed bomb bursts) that the slow-mo button turns into a puzzle instead of a reflex check.

twbG [Gravity-shifting movement prototype](#)

Player redirects their own gravity on the fly and flies toward whichever direction they pick, with interactive objects and collectibles scattered through the level.

Dare to Share [Card-based party game, Android](#)

A prompt-card app for friend groups — pick a category or mix them all, and a full undo/redo history stack lets you step back and forth through the deck.

TECHNICAL SKILLS

Engine: Unity (3+ years)
Languages: C# (primary), C++, Java (basic)
Core frameworks: Zenject (incl. SignalBus), DOTween / PrimeTween, UniTask, Odin Inspector
Systems: Dialogue System & Quest System (Pixel Crushers), Cinemachine, Timeline, Input System
Multiplayer: Client-side prediction, movement reconciliation, lag compensation (custom netcode)
Editor tooling: Custom editor windows/inspectors, ScriptableObject-driven workflows, artist/LD-facing utilities
Other: NaughtyAttributes, Assembly Definitions, Git, Linux, debugging (breakpoints/step-through)
AI-assisted dev: Use LLMs deliberately — speeds up boilerplate and helps sanity-check architecture calls, but I write architecture and code fully unassisted just as often and can tell a bad AI suggestion from a good one

ADDITIONAL EXPERIENCE

Modding & scripting

- Custom HLDS/ReHLDS server plugins (AMX Mod X / Pawn) — built a custom Jailbreak game mode with original menus and events
- Minecraft mod development
- Arduino programming (hobby electronics)

Research

- Published in-depth technical research: "[How to Make 2D Platformer Movement That Feels Good](#)" — a from-scratch breakdown of acceleration curves, jump physics, coyote time, input buffering, and corner correction, with working code (Medium)

EDUCATION

State University of Intelligent Technologies and Telecommunications (Odesa) 2024 - 2028 (expected)
Eötvös Loránd University (ELTE), Budapest — Exchange Program 2026 - 2027

LANGUAGES

Ukrainian — Native **Russian** — Fluent **English** — C1